Evaluation Setup: Deterministic Artifact Tests and Planned Learner Study

Deterministic model suite and fixed inputs

Deterministic model suite and fixed inputs I

All quantitative results derive from small, fully specified discrete models evaluated by the kernels with a fixed seed (20260609). Within the supported Python, NumPy and Matplotlib environment, the model inputs and graph ordering are deterministic; this is a software-environment reproducibility claim, not a cross-hardware bitwise proof.

Deterministic model suite and fixed inputs I

- **Noisy-sensor model** (Bayes and VFE results): two latent states, `present` and `absent`, with prior $D = [0.01, 0.99]$ and likelihood rows $[0.99, 0.05]$ and $[0.01, 0.95]$ over `positive` and `negative` observations. These model parameters are registered as kernel claims, so the prose cannot drift from `src/kernels/examples.py`. This model emits the posterior, evidence probability, negative log-evidence, and free-energy sweep.
- **Informative-sensor model** (EFE decomposition): likelihood rows $[0.9, 0.1]$ and $[0.1, 0.9]$ with observation preference $C = [0.9, 0.1]$. This isolates epistemic value under uncertain versus certain beliefs.
- **Two-action agent**: a neutral-preference sample/commit model with prior $D = [0.5, 0.5]$. The *sample* action preserves the uncertain belief; the *commit* action collapses belief mass toward one state. A sensor-precision sweep over this same model identifies the sample/commit switch.
- **Information examples**: a fair coin, a fair six-sided die, a biased $0.9/0.1$ coin, a one-in-a-hundred event, and a correlated joint distribution with entries $[0.4, 0.1, 0.1, 0.4]$.

Curriculum export and runtime ownership boundary

Curriculum export and runtime ownership boundary I

The curriculum spans the eight strata declared in `manuscript/config.yaml`. The maximal-enumeration floor is 630 skills (achieved: 630), and the current catalog adds practice-and-translation material across all strata. That material specifies opportunities to rehearse model checking, debugging, policy design, implementation review, and application evaluation; it does not establish that learners will acquire those abilities. Skills default to 100 points, one occurrence to completion, and `Quiz` self-reporting so completion is assessment-gated. SkillTree export targets project id `activeInference`.

Reproduction, delivery, and release checks

Reproduction, delivery, and release checks I

The generation pipeline is deterministic and runs from the project root with no Docker daemon and no browser. Only `validate_course_material_sources.py --online` leaves the machine, and only to re-resolve the cited course-material sources:

```
uv run python scripts/build_curriculum.py    # validate the
uv run python scripts/run_kernels.py         # claims, scho
uv run python scripts/build_course_materials.py
uv run python scripts/validate_course_material_sources.py -
uv run python scripts/validate_manuscript_artifacts.py
uv run python scripts/validate_docker_digests.py
uv run python scripts/validate_release_readiness_inventory.
uv run python scripts/export_skilltree.py    # provenance-g
uv run pytest --cov=src --cov-fail-under=90 # coverage gat
```

`validate_docker_digests.py` belongs in that offline list because it reads `docker-compose.yml` as a file rather than talking to a daemon: it fails when any Compose image is named by a mutable tag instead of a `sha256` digest, which is what keeps the live rehearsal below reproducible instead of tag-dependent. Its `--file` flag points the same check at another Compose file.

The remaining checks are live. All of them need a reachable Docker daemon, and the browser-facing ones additionally need the `chrome-devtools-axi` Chrome driver. They fail closed rather than downgrading to the artifact-only dashboard:

Reproduction, delivery, and release checks I

```
./scripts/run_skilltree.sh                          # docker compos
./scripts/run_learning_dashboard.sh                 # SkillTree + s
uv run python scripts/reconcile_live_dependencies.py --chec
uv run python scripts/validate_live_skilltree_dashboard.py
uv run python scripts/validate_skilltree_form_login.py
uv run python scripts/validate_dashboard_console.py --requi
uv run python scripts/validate_full_app_stack.py --app-reac
uv run python scripts/validate_full_app_stack.py --skip-reg
```

Reproduction, delivery, and release checks I

`run_learning_dashboard.sh` regenerates the curriculum, claims, course-material index, figure registry, manuscript artifact audit, figures, and SkillTree export before seeding, so the learner surface is inspected as an actual course interface rather than a static preview. With local auth enabled, the dashboard serves `http://127.0.0.1:8765/api/skilltree/token` as a trusted-client bridge for the official SkillTree display; it never embeds the project client secret in static JavaScript. The dashboard can also run in artifact-only mode with `uv run python scripts/serve_learner_dashboard.py --check`, which verifies that the server, route map, and artifact inventory load without requiring Docker.

Reproduction, delivery, and release checks I

The live gates divide by what they are willing to assert. `reconcile_live_dependencies.py --check-only` reports live prerequisite parity against `output/data/skilltree_project.js` on without writing anything; dropping `--check-only` writes the missing edges into the running service, with `--attempts-per-edge`, `--max-edges`, and `--max-runtime-seconds` bounding a chunked local run. `validate_live_skilltree_dashboard.py` checks the SkillTree and dashboard status endpoints, export-count agreement, artifact completeness, and the trusted-client token exchange by default; its `--require-live-admin`, `--require-live-dependencies`, and `--require-figure-contracts` flags escalate that to live admin skill-count parity, exact live dependency parity, and figure-registry-to-dashboard contract agreement.

Reproduction, delivery, and release checks I

`validate_skilltree_form_login.py` drives the raw SkillTree `FORM` login in a real browser through to the administrator surface and redacts local credentials from its JSON summary. `validate_dashboard_console.py` with `--require-browser --require-dashboard` fails on any dashboard console error other than the allow-listed upstream `Inception` status probe; without those flags it reports a skip when the browser or dashboard is absent.

Reproduction, delivery, and release checks I

`validate_full_app_stack.py` composes the live sequence end to end: Compose bring-up, seeding, dashboard start, dependency reconciliation, the live dashboard checks, the form login, and the console gate. `--app-ready` skips dependency reconciliation and labels its JSON summary app-ready; the flagless run additionally requires every exported prerequisite edge to be present in the live service and labels the summary full-parity. `--skip-regenerate` reuses existing `output/` artifacts instead of rerunning the generation sequence, `--skip-browser` drops the browser stage, and `--keep-running` leaves the dashboard up for manual inspection. Each successful run writes a redacted JSON summary under the local state directory. That evidence is per-run and host-local: it records what one rehearsal observed on one machine, not a public deployment.

Planned learner-outcome study (design only)

Planned learner-outcome study (design only) I

The next learner-outcome step is preregistered in `experiment_plan.yaml` as design-only work, not as a result. The planned pilot uses adult volunteer consent, data minimization, and deidentified study ids before any SkillTree-owned progress export is analyzed. Learner progress, quiz attempts, completion, and ranking remain owned by SkillTree; the local dashboard is not treated as a learner-truth database.

Planned learner-outcome study (design only) II

The planned measurement sequence is baseline concept items, an assigned SkillTree learning path, immediate post-items, delayed-retention items, and an optional transfer prompt scored by a frozen rubric. Active-learning, cognitive-engagement, retrieval-practice, and evidence-centered assessment sources motivate that design envelope, but they do not substitute for data from this curriculum (Freeman et al. 2014; Chi and Wylie 2014; Roediger and Karpicke 2006; Mislevy et al. 2003; Black and Wiliam 1998; Shute 2008; Monib et al. 2025; Rof et al. 2024; Emin et al. 2026).

Planned learner-outcome study (design only) I

Design-based research is the intended study posture because the intervention is a curriculum artifact, a SkillTree delivery configuration, and a measurement workflow that must be studied in context and iterated before stronger outcome language would be appropriate (Design-Based Research Collective 2003; Barab and Squire 2004). Learning-analytics reviews and intervention meta-analyses motivate the outcome-evaluation path but also make privacy, data protection, and instrumentation validity part of the study boundary (Liu et al. 2025; Hernández-Campos et al. 2025; Liu and Khalil 2023; Sclater and Bailey 2015). The protocol will use pilot-and-feasibility reporting and an intervention-description checklist so that implementation readiness is reported separately from definitive learner outcomes (Eldridge et al. 2016; Hoffmann et al. 2014). The analysis plan is within-learner baseline-to-post change with delayed retention reported

Planned learner-outcome study (design only) I

separately after the sample size, item families, exclusions, and rubric are frozen. This manuscript does not report those learner data and makes no efficacy, retention, or transfer claim.

Browser-facing validation path I

For end-to-end browser validation, open the dashboard, confirm the official SkillTree panel mounts the `activeInference` project, switch to the curriculum panel, and open a skill's **Learn**, **Open Subject**, and **Quiz** actions. **Learn** opens the local modular lesson record and should not emit a direct `/subjects/{subjectId}/skills/{skillId}` SkillTree target. **Open Subject** reopens the dashboard with `skillsClientDisplayPath=/subjects/{subjectId}` so the subject learning path, skill list, progress, and rank are exercised inside the official SkillTree display. **Quiz** opens `skillsClientDisplayPath=/subjects/{subjectId}/skills/{skil` so quiz loading, scoring rules, and completion remain SkillTree-owned rather than dashboard-owned.